

INDIAN INSTITUTE OF TECHNOLOGY DELHI

New Delhi – 110016



GIPEDI INTERNSHIP

Experiment Report

Submitted in partial fulfillment of the requirements
for the GIPEDI Internship Programme

Name : Ganesh Mahata
Internship Period : 15 May 2026 – 15 July 2026
Organization : Department of Electrical Engineering
Indian Institute of Technology Delhi
Supervisor : Prof. Subrat Kar
Mentor : Mr. Chandan

July 2026

Contents

1	Experiment 0: Fundamentals of Arduino Interfacing	1
1.1	LED Flash System	1
1.2	7-Segment Display Interfacing	4
1.3	LED Matrix Emoji Display System	7
2	Experiment 1: Sensor Interfacing	10
2.1	Digital Temperature Monitoring System	10
2.2	Temperature Monitoring & Alert System	13
2.3	Weather Forecasting System	15
3	Experiment 2: Calculator Applications	20
3.1	Simple Calculator Using Arduino UNO	20
3.2	BODMAS Method-Based Calculator	24
3.3	Digital Password Lock System	27
4	Experiment 3: Signal Generation and Digital-to-Analogue Conversion	32
4.1	8-Bit Arduino Function Generator using DAC and Op-Amp	32
4.2	Arduino UNO-Based Lissajous Pattern Generator	36
5	Experiment 4: Real-Time Clock Applications	39
5.1	Real-Time Alarm Clock Using RTC and OLED Display	39
5.2	RTC-Based Water Supply Monitoring and Pump Status Display System	43
5.3	Temperature Data Logger Using I ² C Bus	46

Experiment 0: Fundamentals of Arduino Interfacing

1.1 LED Flash System

1.1.1 Problem Statement

The objective of this experiment is to interface an LED with an Arduino Uno and make it blink continuously at a fixed interval.

This experiment helps to understand:

- Digital output pins of Arduino
- Basic Arduino programming structure
- Use of functions in Arduino
- Timing control using `delay()`

1.1.2 Components Used & Pin Connections

Table 1.1: LED Flash Components

Component	Quantity
Arduino UNO	1
LED	1
100 Ω Resistor	1
Breadboard	1
Jumper Wires	Required

Table 1.2: LED Flash Pin Connections

Device	Arduino Pin
LED Positive Terminal	D12
LED Negative Terminal	GND

1.1.3 Software Used

- Arduino IDE
- SimulIDE

1.1.4 Circuit Diagram

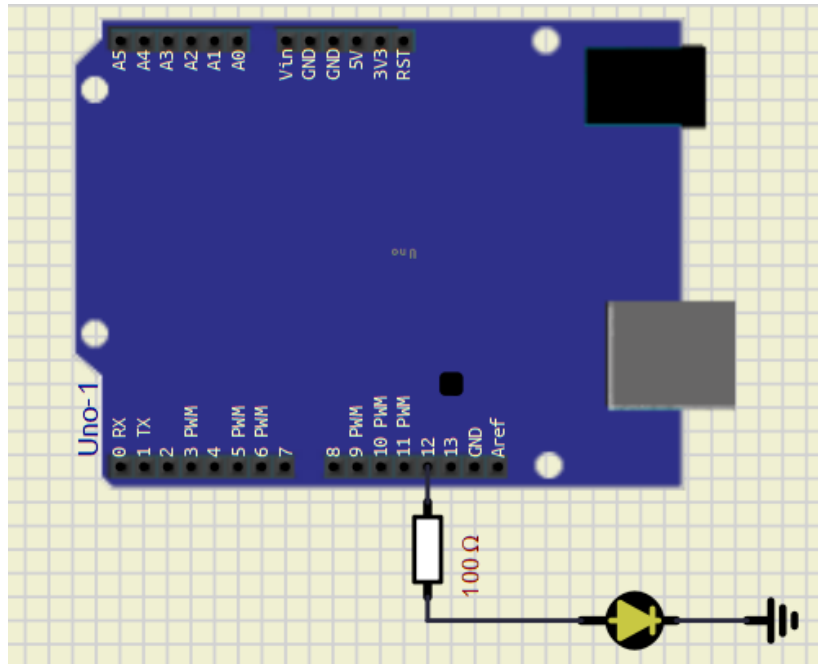


Figure 1.1: Circuit Diagram of LED Flash System

1.1.5 Source Code

The complete Arduino source code for this experiment is available in the project repository.

1.1.6 Working Principle

1. The LED is connected to digital pin D12 of Arduino UNO.
2. In the `setup()` function, pin D12 is configured as an output pin using `pinMode()`.
3. The `loop()` function continuously calls the `toggle_led()` function.
4. Inside the `toggle_led()` function:
 - The LED turns ON using `digitalWrite(HIGH)`
 - Arduino waits for 1 second using `delay(1000)`
 - The LED turns OFF using `digitalWrite(LOW)`
 - Arduino again waits for 1 second
5. This process repeats continuously, causing the LED to blink repeatedly.

1.1.7 What I Learned

- Arduino UNO digital pins give approximately 5V output.
- Recommended current per digital pin is 20mA.
- Maximum current per digital pin is 40mA.
- Typical LED operating current is around 10mA to 20mA.
- LEDs should be connected using a current-limiting resistor.
- Basic LED interfacing with Arduino UNO.

1.1.8 Conclusion

The LED Blinking System using Arduino UNO was successfully implemented and tested.

The LED blinked continuously with a delay of 1 second ON and 1 second OFF. This experiment helped in understanding basic Arduino programming, digital output control, and hardware interfacing.

Experiment 0 – Go Further 1

1.2 7-Segment Display Interfacing

1.2.1 Problem Statement

The objective of this project is to interface a common-cathode 7-segment display with an Arduino Uno and display numbers sequentially.

This experiment helps in understanding:

- 7-segment display interfacing
- Digital output control
- Number display logic
- Multi-pin control using Arduino

1.2.2 Components Used & Pin Connections

Table 1.3: 7-Segment Components

Component	Quantity
Arduino UNO	1
7-Segment Dis-play(Common Cathode)	1
220 Ω Resistors	7
Jumper Wires	Required
Breadboard	1

Table 1.4: 7-Segment Pin Connections

7-Segment Pin	Arduino Pin
a	D2
b	D3
c	D4
d	D5
e	D6
f	D7
g	D8

1.2.3 Software Used

- Arduino IDE
- SimulIDE

1.2.4 Circuit Diagram

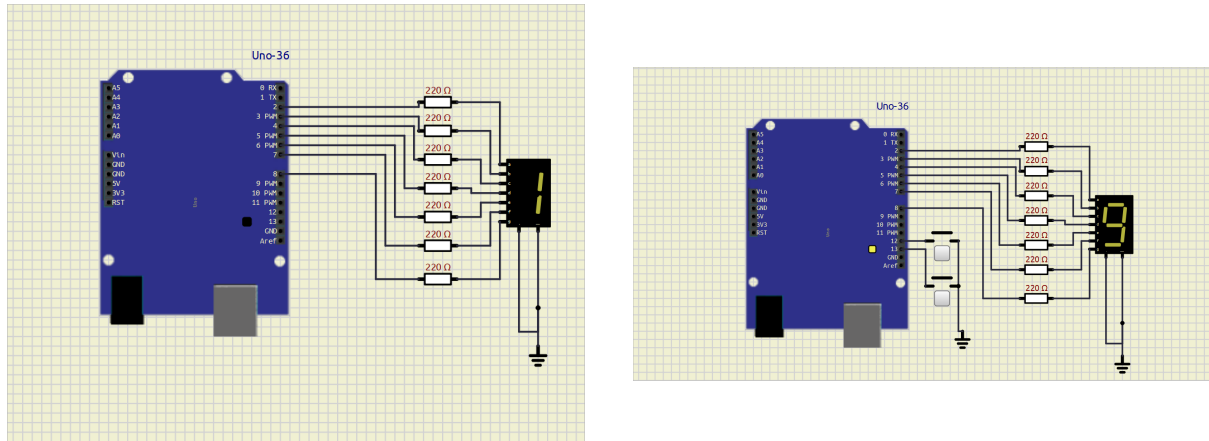


Figure 1.2: Circuit Diagram of 7-Segment Display Interfacing

1.2.5 Source Code

The complete Arduino source code for this experiment is available in the project repository.

1.2.6 Working Principle

1. The 7-segment display segments are connected to Arduino digital pins D2 to D8.
2. Each segment is controlled individually using `digitalWrite()`.
3. Depending on which segments are turned ON or OFF, different numbers are displayed.
4. The Arduino sequentially displays:
 - 0
 - 1
 - 2
 - 3
5. Each number remains visible for 1 second using `delay(1000)`.
6. The sequence repeats continuously.

1.2.7 Learning Outcomes

- Interfacing a 7-segment display with Arduino UNO
- Common cathode display working principle

- Using multiple digital output pins
- Number display logic using segments
- Arduino digital output voltage is approximately 5V
- Recommended current per pin is 20mA
- Typical LED segment current is around 10mA to 20mA

1.2.8 Conclusion

The 7-segment display was successfully interfaced with Arduino UNO.

The display sequentially showed numbers from 0 to 3 by controlling individual segments through Arduino digital pins. This project helped in understanding display interfacing and segment control logic.

Experiment 0 (Go Further 2)

1.3 LED Matrix Emoji Display System

1.3.1 Problem Statement

The objective of this project is to interface an 8×8 LED matrix with Arduino UNO and display different emoji patterns.

This experiment helps in understanding:

- LED matrix interfacing
- Row and column scanning
- Pattern generation
- Multiplexing technique

1.3.2 Components Used & Pin Connections

Table 1.5: LED matrix Components Used and Pin Connections

Components		Pin Connections	
Arduino UNO	1	Row 1	D1
8×8 LED Matrix	1	Row 2	D2
125Ω Resistors	8	Row 3	D3
Breadboard	1	Row 4	D4
Jumper Wires	Required	Row 5	D5
		Row 6	D6
		Row 7	D7
		Row 8	D8
		Column 1	A5
		Column 2	A4
		Column 3	A3
		Column 4	A2
		Column 5	D9
		Column 6	D10
		Column 7	D11
		Column 8	D12

1.3.3 Software Used

- Arduino IDE
- Proteus / SimulIDE

1.3.4 Circuit Diagram

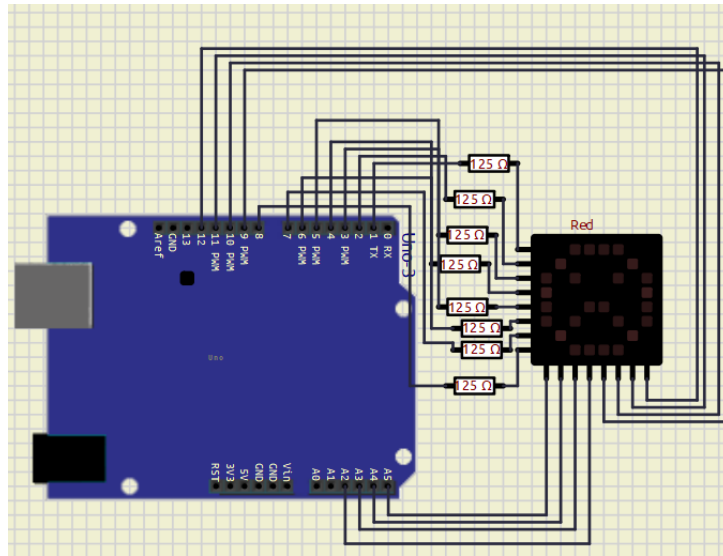


Figure 1.3: Circuit Diagram of LED Matrix Emoji Display System

1.3.5 Source Code

The complete Arduino source code for this experiment is available in the project repository.

1.3.6 Working Principle

1. The 8×8 LED matrix is connected to Arduino UNO using row and column connections.
2. Arduino scans rows and columns rapidly using multiplexing.
3. Binary patterns are stored inside the `emoji` array.
4. The `displayEmoji()` function activates rows one by one and controls columns according to the stored pattern.
5. Different emoji patterns are displayed continuously on the LED matrix.
6. Due to fast scanning, the complete image appears continuously visible.

1.3.7 Learning Outcomes

- Interfacing an LED matrix with Arduino UNO
- Multiplexing technique
- Row and column scanning

- Pattern generation using binary values
- Arduino digital output voltage is approximately 5V
- Recommended current per pin is 20mA
- LED matrix LEDs require current limiting resistors

1.3.8 Conclusion

The LED matrix was successfully interfaced with Arduino UNO.

Different emoji patterns were displayed by controlling rows and columns using multiplexing techniques. This project helped in understanding matrix displays and pattern generation in embedded systems.

Experiment 1: Sensor Interfacing

2.1 Digital Temperature Monitoring System

2.1.1 Problem Statement

The main objective of this project is to design a smart temperature monitoring system using Arduino UNO and a DHT11 temperature sensor.

The system continuously measures room temperature and displays the live temperature value on an SSD1306 OLED display. According to the temperature value, the system automatically controls cooling and heating devices such as a fan and heater.

A push button is also provided to turn the complete system ON or OFF manually.

2.1.2 Components Used & Pin Connections

Table 2.1: Temperature Monitoring Components

Component	Quantity
Arduino UNO	1
DHT11 Temperature Sensor	1
SSD1306 OLED Display	1
Red LED	1
Green LED	1
DC Fan / Motor	1
Heater / Bulb	1
Push Button	1
Jumper Wires	Required

Table 2.2: Temperature Monitoring Pin Connections

Device	Arduino Pin
DHT11 Data Pin	D2
Red LED	D3
Green LED	D4
Push Button	D7
Heater	D8
Fan	D9
OLED SDA	A4
OLED SCL	A5

2.1.3 Software Used

- Arduino IDE
- SimulIDE

2.1.4 Circuit Diagram

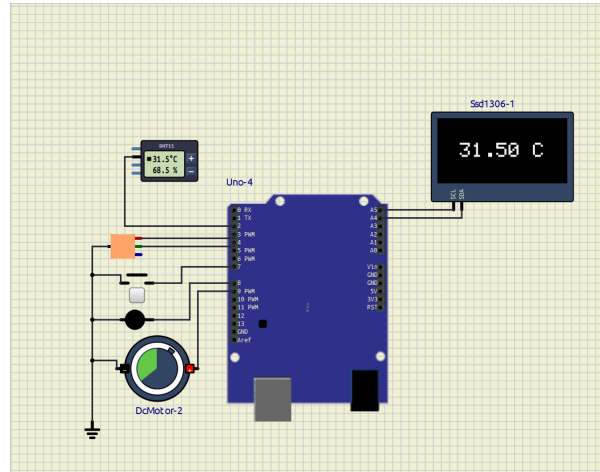


Figure 2.1: Digital Temperature Monitoring System Circuit Diagram

2.1.5 Source Code

The complete Arduino source code for this experiment is available in the project repository.

2.1.6 Working Principle

1. The push button is used to turn the complete system ON or OFF.
2. When the system is ON, the DHT11 sensor continuously measures the room temperature.
3. The measured temperature value is displayed on the SSD1306 OLED display in real time.
4. Arduino UNO continuously compares the measured temperature with the predefined threshold value of 28°C.
5. If the temperature becomes greater than or equal to 28°C:
 - Red LED turns ON.
 - Fan starts running automatically.
 - Heater turns OFF.

6. If the temperature becomes lower than 28°C:
 - Green LED turns ON.
 - Heater turns ON automatically.
 - Fan turns OFF.
7. The OLED display continuously updates the live temperature reading.
8. When the push button is pressed again:
 - The complete system turns OFF.
 - OLED display becomes blank.
 - LEDs turn OFF.
 - Fan and heater stop working.

2.1.7 Conclusion

The Digital Temperature Monitoring System using Arduino UNO and the DHT11 temperature sensor was successfully implemented and tested in SimulIDE.

The system continuously monitored room temperature and automatically controlled the fan and heater according to the measured temperature. The SSD1306 OLED display successfully showed the live temperature reading, while the push button provided convenient ON/OFF control of the complete system.

This project helped to understand the following:

- Arduino interfacing
- Sensor interfacing
- OLED display handling
- Embedded system automation
- Temperature-based control systems

Experiment 1 (Go Further 1):

2.2 Temperature Monitoring & Alert System

2.2.1 Problem Statement

This project monitors temperature using a DHT11 sensor and activates a buzzer when the temperature exceeds the threshold value of 50°C. The system is designed using Arduino UNO in SimulIDE.

2.2.2 Components Used & Pin Connections

Table 2.3: Emergency Alert System Components

Component	Quantity
Arduino UNO	1
DHT11 Temperature Sensor	1
Buzzer	1
Jumper Wires	Required

Table 2.4: Emergency Alert System Pin Connections

Device	Arduino Pin
DHT11 Data Pin	D1
Buzzer	D8
DHT11 VCC	5V
DHT11 GND	GND

2.2.3 Software Used

- Arduino IDE
- SimulIDE

2.2.4 Circuit Diagram

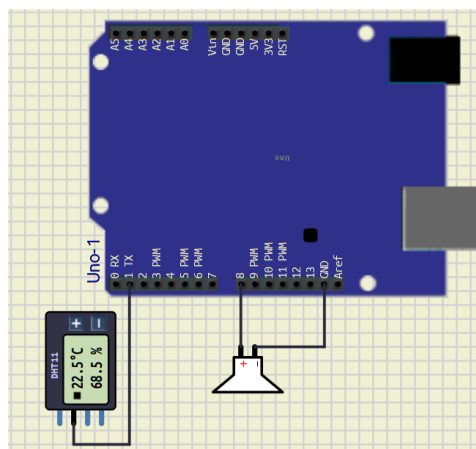


Figure 2.2: Temperature Monitoring-Emergency Alert System Circuit Diagram

2.2.5 Source Code

The complete Arduino source code for this experiment is available in the project repository.

2.2.6 Working Principle

1. The DHT11 temperature sensor continuously measures the surrounding temperature.
2. Arduino UNO continuously reads the temperature value from the DHT11 sensor.
3. The measured temperature is compared with the predefined threshold value of 50°C.
4. If the temperature becomes greater than or equal to 50°C:
 - The buzzer turns ON.
 - A temperature alert is generated.
5. If the temperature remains below 50°C:
 - The buzzer remains OFF.
6. The monitoring process repeats continuously with a delay of one second between consecutive readings.

2.2.7 Output

- Buzzer remains OFF below 50°C.
- Buzzer turns ON at or above 50°C.
- System performs real-time temperature monitoring.

2.2.8 Conclusion

The Temperature Monitoring-Emergency Alert System using Arduino UNO and the DHT11 temperature sensor was successfully implemented and tested in SimulIDE.

The system continuously monitored the surrounding temperature and activated the buzzer whenever the measured temperature exceeded the predefined threshold value of 50°C. This project demonstrated a simple and effective temperature-based emergency alert mechanism suitable for embedded monitoring applications.

Experiment 1 (Go Further 2):

2.3 Weather Forecasting System

2.3.1 Problem Statement

This project monitors the environmental temperature and humidity using a DHT-11 sensor and predicts weather conditions based on temperature readings.

The system uses an RGB LED to indicate different weather conditions:

- Extreme Hot Weather
- Normal Weather
- Cold Weather

A DS1307 RTC module provides real-time clock information, while an SSD1306 OLED display continuously shows:

- Current Time
- Temperature
- Humidity
- Weather Status

A push button is also provided as a master power control to turn the entire system ON and OFF.

2.3.2 Components Used & Pin Connections

Table 2.5: Weather Forecasting System Components

Component	Quantity
Arduino UNO	1
DHT11 Sensor	1
DS1307 RTC Module	1
SSD1306 OLED Display	1
RGB LED	1
Push Button	1
100 Ω Resistors	3
Jumper Wires	Required
Breadboard	1

Table 2.6: Weather Forecasting Pin Connections

Device	Arduino Pin
DHT11 Data Pin	D7
Red LED	D2
Green LED	D3
Blue LED	D5
Power Button	D8
OLED SDA	A4
OLED SCL	A5
RTC SDA	A4
RTC SCL	A5

2.3.3 RGB LED Connections

Table 2.7: RGB LED Connections

RGB Pin	Connection
Red Pin	D2 through 100Ω resistor
Green Pin	D3 through 100Ω resistor
Blue Pin	D5 through 100Ω resistor
Common Cathode	GND

2.3.4 Software Used

- Arduino IDE
- SimulIDE

2.3.5 Circuit Diagram

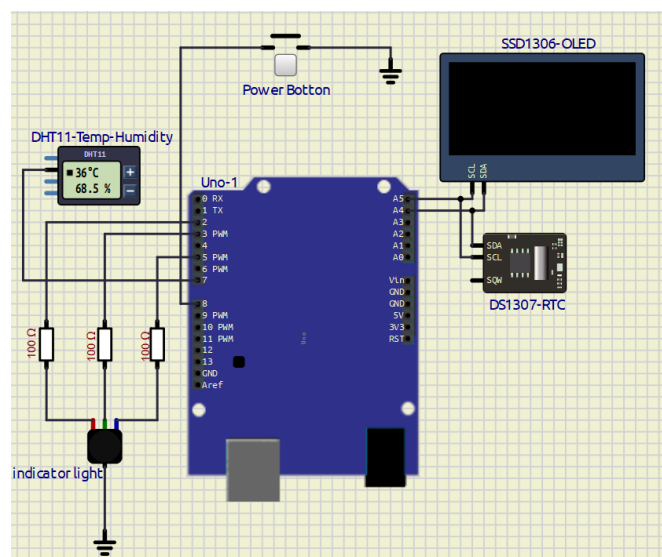


Figure 2.3: Weather Forecasting System Circuit Diagram

2.3.6 Source Code

The complete Arduino source code for this experiment is available in the project repository.

2.3.7 Working Principle

1. The system starts with a blank OLED display.
2. Pressing the power button:

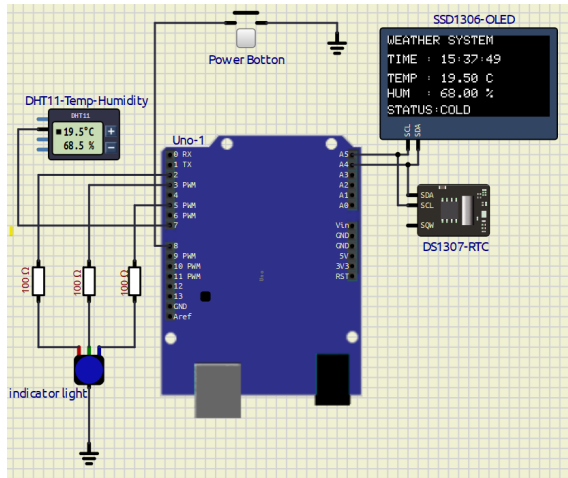
- Turns the system ON.
 - Displays `POWER ON` on the OLED display.
3. The DHT11 sensor continuously measures:
- Temperature
 - Humidity
4. The DS1307 RTC module continuously provides the real-time clock.
5. The OLED display continuously shows:
- Current Time
 - Temperature
 - Humidity
 - Weather Status
6. The RGB LED indicates the weather condition:
- Red LED $\rightarrow$ Extreme Hot Weather
 - Green LED $\rightarrow$ Normal Weather
 - Blue LED $\rightarrow$ Cold Weather
7. Pressing the power button again:
- Displays `POWER OFF`
 - Turns OFF all LEDs
 - OLED returns to a blank screen

2.3.8 Temperature Indicator Logic

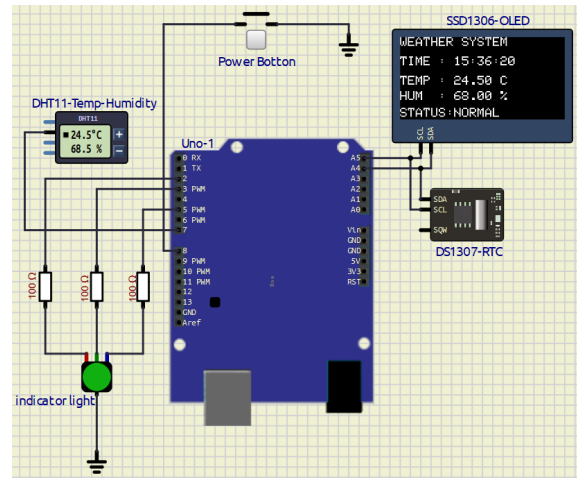
Table 2.8: Temperature Indicator Logic

Temperature	LED Status	Weather Condition
$> 35^{\circ}\text{C}$	Red LED ON	Extreme Hot
$20^{\circ}\text{C} - 35^{\circ}\text{C}$	Green LED ON	Normal
$< 20^{\circ}\text{C}$	Blue LED ON	Cold

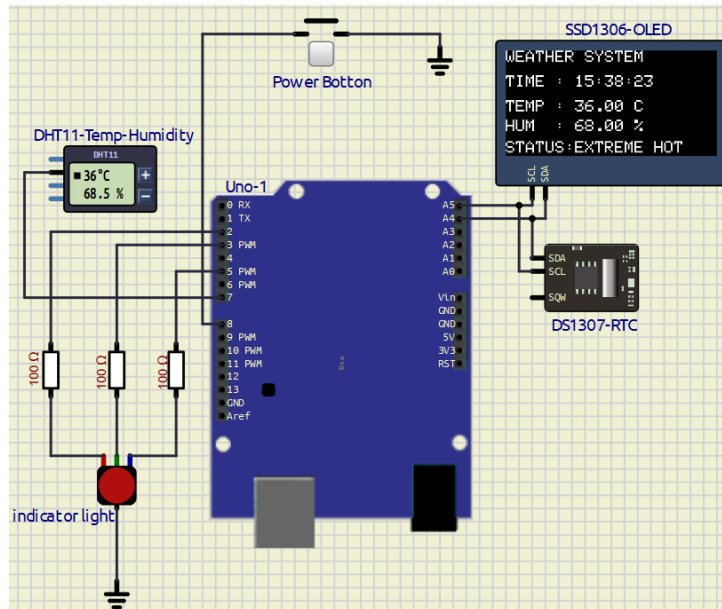
2.3.9 Output



(a) Cold Weather



(b) Normal Weather



(c) Extreme Hot Weather

Figure 2.4: Output of the Weather Forecasting System under different weather conditions

2.3.10 Conclusion

The Weather Forecasting System using Arduino UNO, DHT11 sensor, DS1307 RTC module, SSD1306 OLED display, RGB LED, and push-button control was successfully implemented and tested in SimulIDE.

The system correctly monitored weather conditions, displayed real-time environmental information, and visually indicated temperature conditions using RGB LEDs.

During this experiment, the following concepts were learned:

- Sensor interfacing
- OLED interfacing
- RTC interfacing
- RGB LED control
- Push-button logic
- Embedded system design

Experiment 2: Calculator Applications

This chapter presents the implementation of a simple calculator using Arduino UNO, a 4×4 keypad, and an SSD1306 OLED display. The project demonstrates user input through a keypad, real-time arithmetic processing, and graphical output on an OLED display.

3.1 Simple Calculator Using Arduino UNO

3.1.1 Problem Statement

This project implements a simple calculator using Arduino UNO, a 4×4 keypad, and an SSD1306 OLED display. The calculator performs basic arithmetic operations such as addition, subtraction, multiplication, and division. The calculated result is displayed on the OLED screen in real time.

3.1.2 Components Used & Pin Connections

Table 3.1: Calculator Components

Component	Quantity
Arduino UNO	1
4×4 Keypad	1
SSD1306 OLED Display	1
Jumper Wires	Required
Breadboard	1

Table 3.2: Keypad Pin Connections

Keypad Pin	Arduino Pin
R1	D1
R2	D2
R3	D3
R4	D4
C1	D5
C2	D6
C3	D7
C4	D8

3.1.3 OLED Display Connections

Table 3.3: SSD1306 OLED Display Connections

OLED Pin	Arduino Pin
VCC	5V
GND	GND
SDA	A4
SCL	A5

3.1.4 Keypad Operator Mapping

Table 3.4: Keypad Operator Mapping

Symbol	Operation
A	Addition (+)
B	Subtraction (-)
C	Multiplication (*)
D	Division (/)
#	All Clear (AC)

3.1.5 Software Used

- Arduino IDE
- SimulIDE

3.1.6 Circuit Diagram

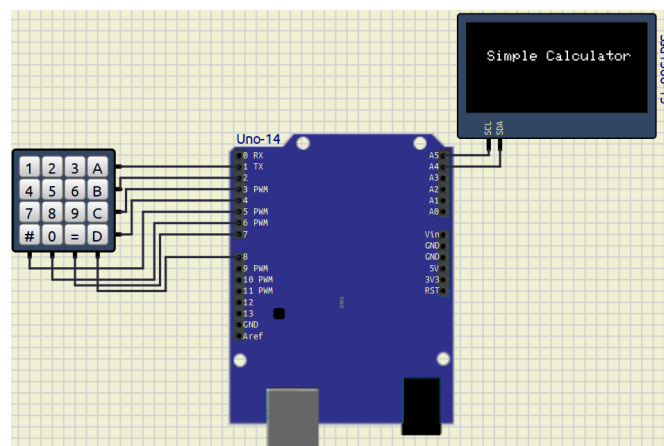


Figure 3.1: Simple Calculator Using Arduino UNO

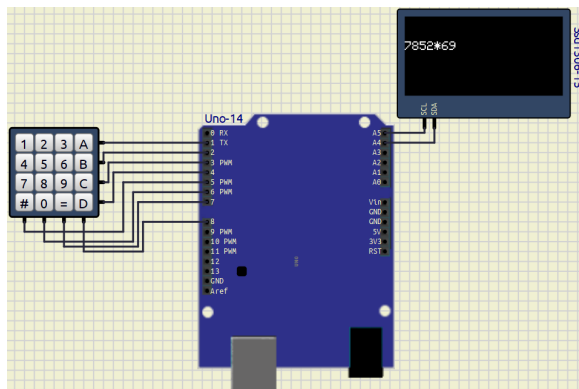
3.1.7 Source Code

The complete Arduino source code for this experiment is available in the project repository.

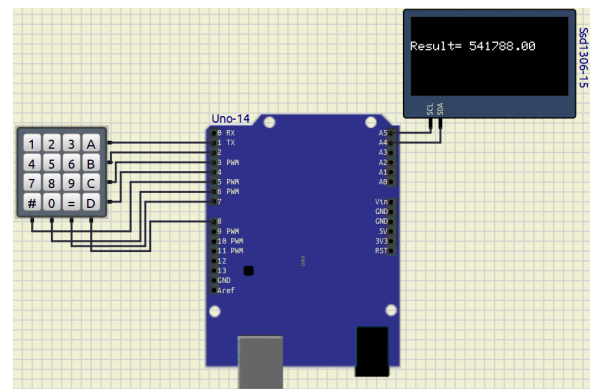
3.1.8 Working Principle

1. The 4×4 keypad is used to enter numbers and arithmetic operators.
2. Arduino UNO continuously scans the keypad to detect key presses.
3. The entered numeric values are stored as operands.
4. The selected operator determines the arithmetic operation to be performed.
5. When the '=' key is pressed:
 - Arduino performs the requested calculation.
 - The computed result is displayed on the SSD1306 OLED display.
6. Pressing the '#' key clears all previous inputs and resets the calculator.
7. The calculator then becomes ready for the next calculation.

3.1.9 Output



(a) Arithmetic Operation



(b) Calculation Result

Figure 3.2: Calculator Operation and Output

3.1.10 Conclusion

The Simple Calculator using Arduino UNO, a 4×4 Keypad, and an SSD1306 OLED Display was successfully implemented and tested.

The system successfully performs basic arithmetic operations, including addition, subtraction, multiplication, and division. User input is received through the keypad, and the calculated result is displayed on the OLED screen in real time.

During this experiment, the following concepts were learned:

- Matrix keypad interfacing
- OLED display interfacing
- User input processing
- Arithmetic operation implementation
- Embedded system programming

One challenge encountered during the implementation was mapping the keypad keys to their corresponding arithmetic operations. This issue was resolved by referring to the official Arduino documentation and open-source community resources.

Experiment 2 (Go Further 1):

3.2 BODMAS Method-Based Calculator

3.2.1 Problem Statement

The objective of this project is to design a calculator using Arduino UNO, an SSD1306 OLED display, and a 5×4 matrix keypad that performs arithmetic calculations according to the BODMAS (Bracket, Order, Division, Multiplication, Addition, and Subtraction) rule.

The calculator evaluates mathematical expressions entered through the keypad and displays the calculated result on the OLED display. Additional features such as history, backspace, and expression clearing improve the usability of the system.

3.2.2 Components Used & Pin Connections

Table 3.5: Components Used

Component	Qty
Arduino UNO	1
SSD1306 OLED Display	1
5×4 Matrix Keypad	1
Breadboard	1
Jumper Wires	Req.

Table 3.6: Keypad Connections

Pin	Arduino
R1–R5	D2–D6
C1–C4	D7–D10

Table 3.7: OLED Connections

OLED Pin	Arduino
SDA	A4
SCL	A5

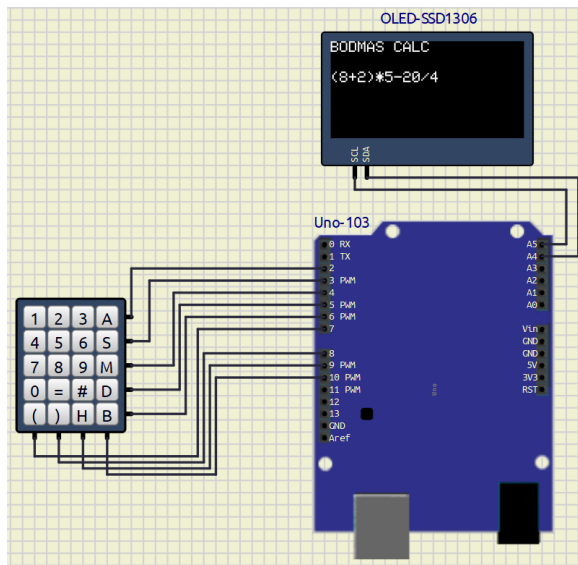
Table 3.8: Keypad Functions

Key	Function
A	Addition (+)
S	Subtraction (-)
M	Multiplication (*)
D	Division (/)
=	Calculate
#	Clear
B	Backspace
H	History

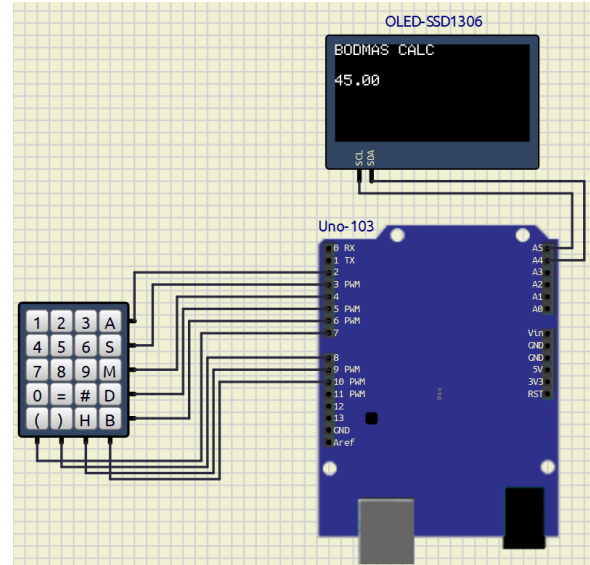
3.2.3 Software Used

- Arduino IDE
- SimulIDE

3.2.4 Circuit Diagram



(a) Circuit Diagram - 1



(b) Circuit Diagram - 2

Figure 3.3: Circuit Diagram of the BODMAS Method Based Calculator

3.2.5 Source Code

The complete Arduino source code for this experiment is available in the project repository.

3.2.6 Working Principle

1. The user enters a mathematical expression using the 5×4 matrix keypad.
2. Arduino UNO continuously scans the keypad and receives the entered input.
3. The entered mathematical expression is displayed on the SSD1306 OLED display in real time.
4. When the '=' key is pressed, Arduino evaluates the complete expression according to the BODMAS rule.
5. The calculated result is displayed on the OLED display.
6. Previous calculations are automatically stored in the history for future reference.
7. The '#' key clears the current expression, while the 'B' key removes the last entered character (Backspace).
8. The 'H' key displays the calculation history stored in memory.

3.2.7 Output

Example Calculation

Table 3.9: Example BODMAS Calculation

Input Expression	$(8+2)*5-20/4$
Calculated Result	45

The calculator successfully evaluated the mathematical expression according to the BODMAS rule and displayed the correct result on the OLED display.

3.2.8 Conclusion

The BODMAS Method-Based Calculator using Arduino UNO was successfully implemented and tested.

The calculator correctly evaluated mathematical expressions according to the BODMAS rule and displayed the calculated results on the SSD1306 OLED display. The addition of history, backspace, and expression-clearing features enhanced the calculator's usability and demonstrated efficient implementation of expression parsing in embedded systems.

During this experiment, the following concepts were learned:

- Matrix keypad interfacing
- OLED display interfacing
- Expression parsing
- BODMAS rule implementation
- Embedded system programming

Experiment 2 (Go Further 2):

3.3 Digital Password Lock System

3.3.1 Project Name

Digital Password Lock System using Arduino UNO, OLED Display, Keypad, LEDs, and Buzzer

3.3.2 Problem Statement

The objective of this project is to design and implement a Digital Password Lock System using Arduino UNO that provides secure password-based authentication.

The system allows a user to create a four-character password, authenticate access using the stored password, provide visual and audio feedback for successful and failed login attempts, and support secure password reset functionality. A three-attempt security mechanism is also implemented to improve system security.

3.3.3 Components Used & Pin Connections

Table 3.10: Password Lock Components

Component	Quantity
Arduino UNO	1
SSD1306 OLED Display (I2C)	1
4×5 Matrix Keypad	1
Red LED	1
Green LED	1
Buzzer	1
Breadboard	1
Connecting Wires	As Required
USB Cable	1

Table 3.11: LED & Buzzer Connections

Component	Arduino Pin
Buzzer	D11
Green LED	D12
Red LED	D13

3.3.4 OLED Display Connections

Table 3.12: OLED Display Connections

OLED Pin	Arduino Pin
VCC	5V
GND	GND
SDA	A4
SCL	A5

3.3.5 Keypad Connections

Table 3.13: 4×5 Matrix Keypad Connections

Keypad Pin	Arduino Pin
R1	D2
R2	D3
R3	D4
R4	D5
R5	D6
C1	D7
C2	D8
C3	D9
C4	D10

3.3.6 Software Used

- Arduino IDE
- SimulIDE

3.3.7 Circuit Diagram

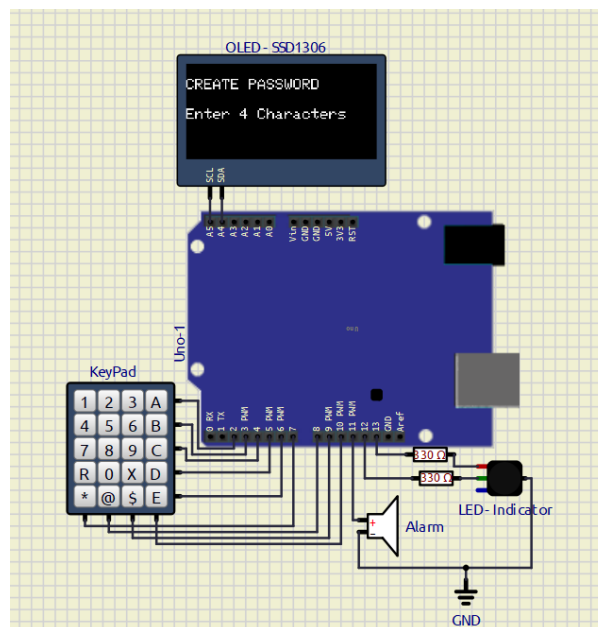


Figure 3.4: Digital Password Lock System Circuit Diagram

3.3.8 Source Code

The complete Arduino source code for this experiment is available in the project repository.

3.3.9 Working Principle

1. When the system is powered ON, the user is prompted to create a four-character administrator password.
2. The entered password is securely stored in the Arduino memory and is used for future authentication.
3. During login, the user enters the password using the 4×5 matrix keypad.
4. Each key press is represented by an asterisk (*) on the OLED display to protect the password from being viewed.
5. If the entered password matches the stored password:
 - Access is granted.
 - The green LED turns ON.
 - A success beep is generated using the buzzer.
6. If an incorrect password is entered:
 - Access is denied.
 - The red LED turns ON.
 - An error beep is generated.
 - The remaining login attempts are displayed on the OLED screen.
7. After three consecutive incorrect password attempts:
 - The system enters Alarm Mode.
 - The red LED blinks continuously.
 - The buzzer sounds continuously.
 - The system remains locked until reset.
8. The password can be reset securely by pressing the R key and entering the existing password before creating a new password.

3.3.10 System Output

Table 3.14: Digital Password Lock System Output

System State	OLED Display
System Startup Enter 4 Characters	SET ADMIN PASS
Password Saved System Ready	PASSWORD SAVED
User Login Unlock The System	ENTER PASSWORD
Correct Password	ACCESS GRANTED
Wrong Password LEFT: 2	ACCESS DENIED
Alarm Mode PRESS R RESET	SYSTEM LOCKED
Password Reset Enter Old Pass	RESET PASSWORD
Reset Successful Set New Pass	CORRECT PASSWORD

3.3.11 Problems Faced

- OLED display address mismatch (0x3C / 0x3D).
- Incorrect SDA and SCL wiring.
- Keypad row and column mapping errors.
- Undefined LED pin names during coding.
- OLED text overflow due to long messages.
- Alarm mode creating an infinite loop during debugging.
- Password reset functionality required additional verification logic.
- Synchronizing keypad input with OLED display updates.

3.3.12 Conclusion

The Digital Password Lock System using Arduino UNO was successfully implemented and tested.

The system provides secure password-based authentication using a matrix keypad and OLED display. Visual feedback through LEDs, audio indication using a buzzer, and a three-attempt security mechanism improve both usability and system security. The password reset feature further enhances flexibility while maintaining secure access control.

During this experiment, the following concepts were learned:

- Matrix keypad interfacing
- OLED display interfacing
- Password authentication
- Embedded security systems
- User access control
- Alarm system implementation

Experiment 3: Signal Generation and Digital-to-Analogue Conversion

This chapter presents the implementation of an 8-bit Arduino Function Generator using an R-2R DAC, operational amplifier, and RC filter. The experiment demonstrates the generation of sine, square, and triangle waveforms using Direct Digital Synthesis (DDS), along with frequency and amplitude control through potentiometers.

4.1 8-Bit Arduino Function Generator using DAC and Op-Amp

4.1.1 Project Name

8-Bit Arduino Function Generator using DAC and Op-Amp

4.1.2 Problem Statement

This experiment demonstrates waveform generation using Arduino UNO, an 8-bit R-2R DAC, operational amplifier, and RC filter.

The system generates sine, square, and triangle waveforms using the Direct Digital Synthesis (DDS) technique. Frequency and amplitude of the generated signals can be adjusted using potentiometers, while an SSD1306 OLED display provides real-time information about the selected waveform and control parameters.

4.1.3 Components Used & Pin Connections

Table 4.1: Function Generator Components

Component	Quantity
Arduino UNO	1
8-Bit R-2R DAC	1
SSD1306 OLED Display	1
Operational Amplifier	1
10k Potentiometer	2
Push Switch	4
Resistors	Multiple
Capacitor	1
Oscilloscope	1

Table 4.2: Control Connections

Device	Arduino Pin
Power Switch	D8
Sine Wave Switch	D9
Square Wave Switch	D10
Triangle Wave Switch	D11
Frequency Potentiometer	A0
Amplitude Potentiometer	A1
OLED SDA	A4
OLED SCL	A5

4.1.4 DAC Connections

Table 4.3: 8-Bit DAC Connections

DAC Pin	Arduino Pin
D0	D0
D1	D1
D2	D2
D3	D3
D4	D4
D5	D5
D6	D6
D7	D7

4.1.5 Software Used

- Arduino IDE
- SimulIDE

4.1.6 Circuit Diagram

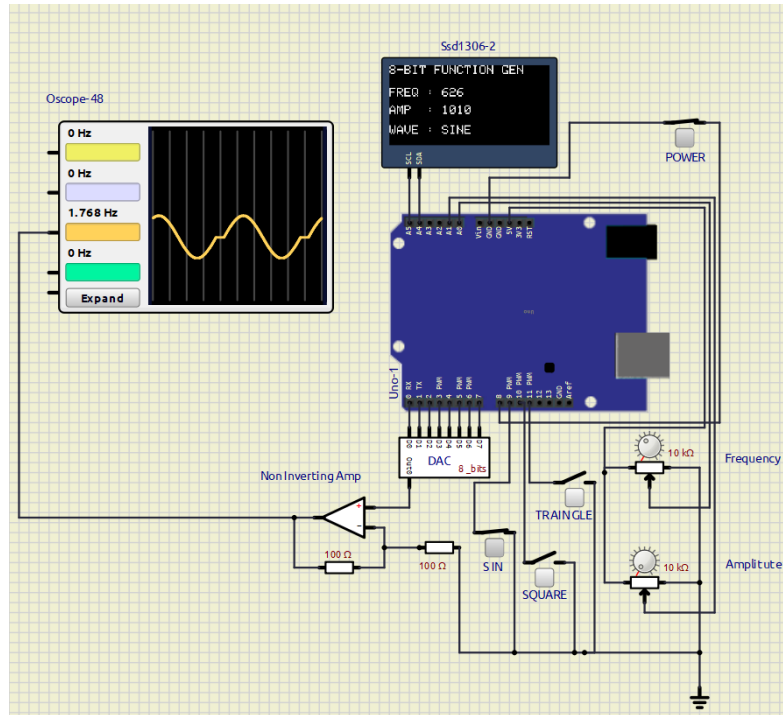


Figure 4.1: 8-Bit Arduino Function Generator Circuit Diagram

4.1.7 Source Code

The complete Arduino source code for this experiment is available in the project repository.

4.1.8 Working Principle

1. Arduino UNO continuously reads the frequency and amplitude values from two potentiometers.
2. The user selects the desired waveform (Sine, Square, or Triangle) using dedicated push switches.
3. A 256-sample sine lookup table is stored in memory for efficient sine wave generation using the Direct Digital Synthesis (DDS) technique.
4. Arduino outputs 8-bit digital data through PORTD (D0–D7).
5. The R-2R DAC converts the digital values into corresponding analog voltage levels.
6. The operational amplifier buffers the DAC output and improves signal stability.
7. An RC low-pass filter smooths the staircase DAC output to produce a cleaner analog waveform.

8. The generated waveform is observed on an oscilloscope, while the OLED display continuously shows the selected waveform, frequency, and amplitude values.

4.1.9 Output

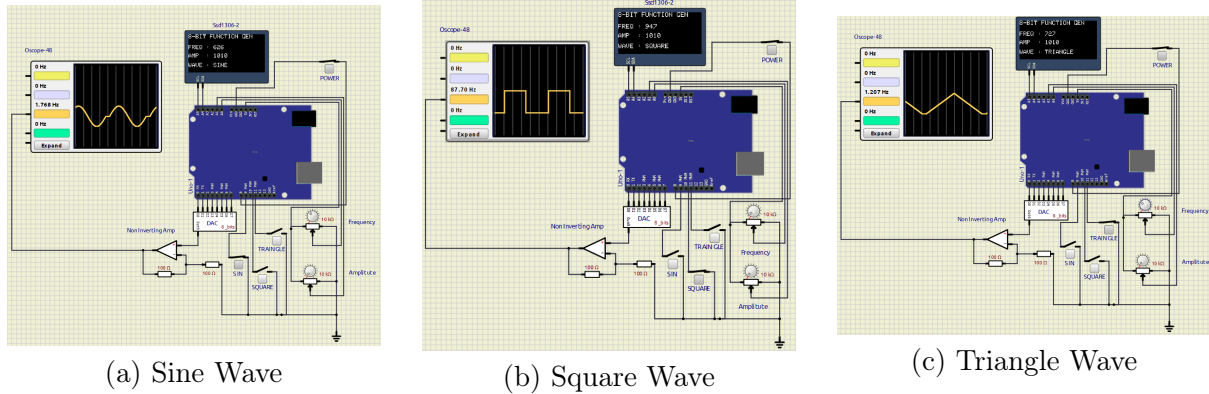


Figure 4.2: Generated Waveforms of the 8-Bit Arduino Function Generator

4.1.10 Learning Outcomes

- Understanding of Digital-to-Analog Conversion (DAC)
- Direct Digital Synthesis (DDS) waveform generation
- High-speed digital output using PORTD
- Frequency control using timing delay
- Waveform generation using lookup tables
- OLED interfacing with Arduino
- Signal conditioning using operational amplifiers and RC filters

4.1.11 Conclusion

The 8-Bit Arduino Function Generator using an R-2R DAC, operational amplifier, and RC filter was successfully designed and simulated using Arduino UNO.

The system generated sine, square, and triangle waveforms using the Direct Digital Synthesis (DDS) technique. Adjustable frequency and amplitude controls, together with OLED-based status monitoring, provided a practical understanding of waveform generation, DAC operation, signal conditioning, and embedded system programming.

Experiment 3 (Go Further 2):

4.2 Arduino UNO-Based Lissajous Pattern Generator

4.2.1 Problem Statement

This experiment demonstrates the generation and visualisation of Lissajous patterns using two sinusoidal signals and an SSD1306 OLED display.

The project helps in understanding frequency ratios, phase difference, analogue signal sampling, coordinate mapping, and graphical visualisation using Arduino UNO.

Different geometric patterns are generated when two sine waves having different frequencies and phase shifts are simultaneously applied to the X-axis and Y-axis of the display system.

4.2.2 Components Used & Pin Connections

Table 4.4: Lissajous Pattern Generator Components

Component	Quantity
Arduino UNO	1
SSD1306 OLED Display	1
Sine Wave Generator	2
Connecting Wires	Multiple
Oscilloscope (Optional)	1

Table 4.5: Signal Connections

Device	Arduino Pin
X Signal Input	A0
Y Signal Input	A1
OLED SDA	A4
OLED SCL	A5

4.2.3 Software Used

- Arduino IDE
- SimulIDE

4.2.4 Circuit Diagram

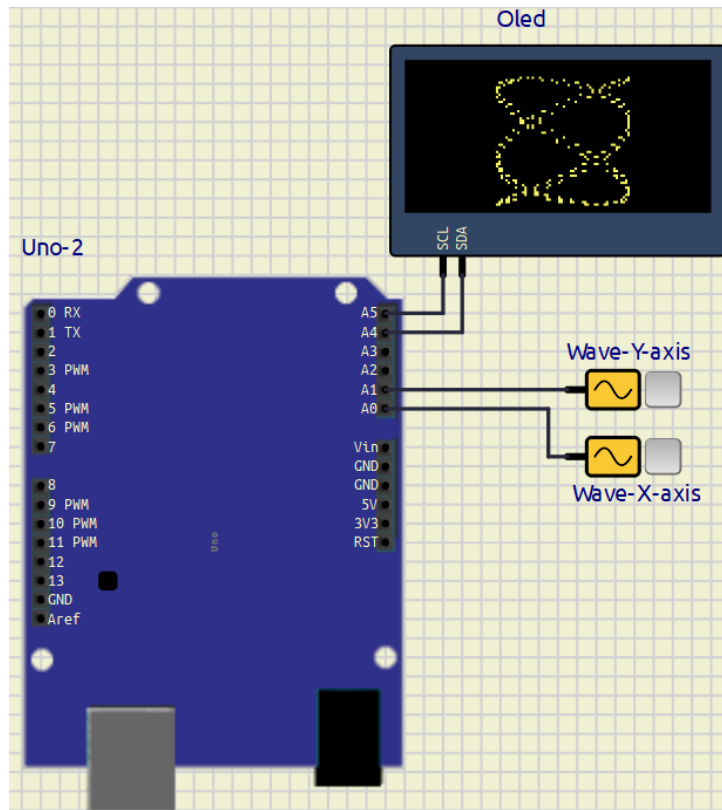


Figure 4.3: Arduino UNO-Based Lissajous Pattern Generator Circuit Diagram

4.2.5 Source Code

The complete Arduino source code for this experiment is available in the project repository.

4.2.6 Working Principle

1. Two sinusoidal input signals are applied to the Arduino UNO through analog input pins A0 and A1.
2. Arduino continuously samples both analog signals using its built-in Analog-to-Digital Converter (ADC).
3. The signal connected to A0 determines the horizontal (X-axis) coordinate, while the signal connected to A1 determines the vertical (Y-axis) coordinate.
4. The sampled analog values are mapped to the OLED display coordinates.
5. Each pair of X-Y coordinates is plotted as a pixel on the SSD1306 OLED display.
6. Continuous sampling and plotting of these coordinate pairs generates a Lissajous pattern.

7. The shape of the generated pattern depends on the frequency ratio and phase difference between the two sinusoidal input signals.
8. The OLED display continuously refreshes to visualize the changing Lissajous figures in real time.

4.2.7 Output

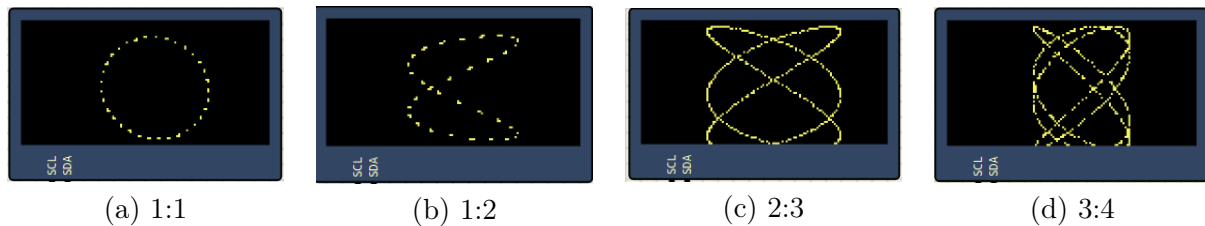


Figure 4.4: Different Lissajous Patterns Generated Using Various Frequency Ratios

4.2.8 Conclusion

The Arduino UNO Based Lissajous Pattern Generator was successfully implemented using dual sinusoidal inputs and an SSD1306 OLED display.

The system continuously sampled two analog signals, mapped them to X-Y coordinates, and generated various Lissajous figures in real time. The experiment provided a practical understanding of frequency ratio, phase difference, analog signal sampling, graphical visualization, and embedded graphics programming.

During this experiment, the following concepts were learned:

- Analog-to-Digital Conversion (ADC)
- Signal sampling techniques
- Coordinate mapping
- Lissajous pattern generation
- OLED graphics programming
- Real-time signal visualization

Experiment 4: Real-Time Clock Applications

This chapter presents the implementation of a Real-Time Alarm Clock using Arduino UNO, a DS1307 RTC module, an SSD1306 OLED display, push buttons, and a buzzer. The project demonstrates real-time clock interfacing, alarm setting, user interaction through push buttons, and embedded timing applications.

5.1 Real-Time Alarm Clock Using RTC and OLED Display

5.1.1 Project Name

Real-Time Alarm Clock Using Arduino UNO, DS1307 RTC Module, SSD1306 OLED Display and Push Buttons

5.1.2 Problem Statement

The objective of this project is to interface a DS1307 Real-Time Clock (RTC) module with Arduino UNO using the I²C communication protocol and display the current date and time on an SSD1306 OLED display.

The system also provides an alarm-setting mechanism using push buttons. Users can configure the alarm hour, minute, and second through dedicated buttons. When the current RTC time matches the preset alarm time, the buzzer is continuously activated until the time changes.

5.1.3 Components Used & Pin Connections

Table 5.1: Alarm Clock Components

Component	Quantity
Arduino UNO	1
DS1307 RTC Module	1
SSD1306 OLED Display	1
Push Buttons	3
Buzzer	1
Breadboard	1
Jumper Wires	Required

Table 5.2: Push Button Connections

Button	Pin	Function
P-adj	D1	Change Alarm Setting Mode
P+ve	D2	Increase Value
P-ve	D3	Decrease Value

5.1.4 RTC, OLED and Buzzer Connections

Table 5.3: RTC, OLED and Buzzer Connections

Device	Arduino Pin
Buzzer (+)	D12
Buzzer (-)	GND
RTC VCC	5V
RTC GND	GND
RTC SDA	A4
RTC SCL	A5
OLED VCC	5V
OLED GND	GND
OLED SDA	A4
OLED SCL	A5

5.1.5 Button Operation

Table 5.4: P-adj Button Operation

Press	Operation
1st	Activate Alarm Setting Mode
2nd	Set Hour
3rd	Set Minute
4th	Set Second
5th	Exit Alarm Setting Mode

- **P+ve Button:** Increases the selected value.
- **P-ve Button:** Decreases the selected value.

5.1.6 Software Used

- Arduino IDE
- SimulIDE

5.1.7 Circuit Diagram

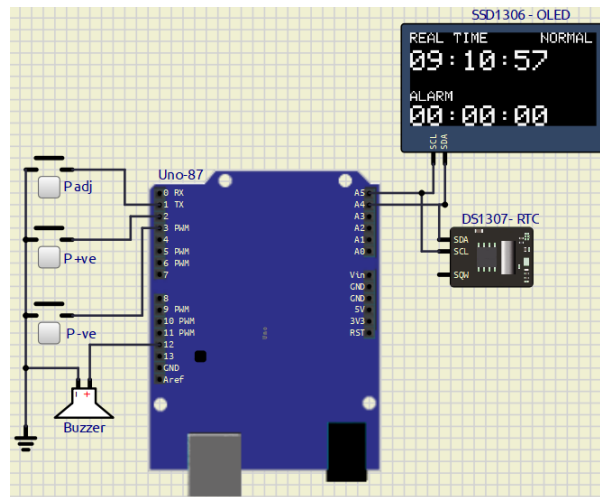


Figure 5.1: Real-Time Alarm Clock Circuit Diagram

5.1.8 Source Code

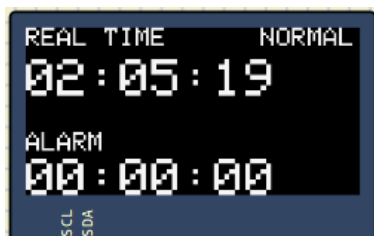
The complete Arduino source code for this experiment is available in the project repository.

5.1.9 Working Principle

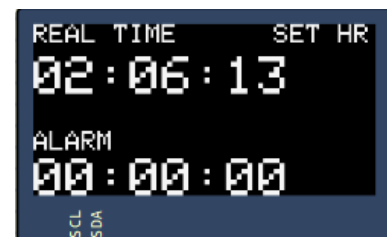
1. The DS1307 RTC module continuously maintains the current date and time using its internal clock.
2. Arduino UNO communicates with the RTC module through the I²C interface and reads the current time periodically.
3. The SSD1306 OLED display continuously shows:
 - Current Time
 - Alarm Time
 - Current Alarm Setting Mode
4. The **P-adj** button is used to change the alarm configuration mode.

5. The **P+ve** and **P-ve** buttons increase or decrease the selected alarm parameter (Hour, Minute, or Second).
6. The configured alarm hour, minute, and second are stored in memory.
7. When the current RTC time matches the configured alarm time:
 - The Arduino activates the buzzer.
 - The buzzer sounds continuously to notify the user.
8. The alarm automatically stops when the current time changes beyond the configured alarm time.

5.1.10 Output



(a) Real-Time Clock Display



(b) Alarm Setting Mode

Figure 5.2: Output of the Real-Time Alarm Clock System

5.1.11 Conclusion

The Real-Time Alarm Clock using Arduino UNO, DS1307 RTC Module, SSD1306 OLED Display, push buttons, and a buzzer was successfully implemented and tested.

The system accurately displayed the current time, allowed users to configure alarm settings, and activated the buzzer when the current RTC time matched the preset alarm time. This experiment provided practical experience in real-time clock interfacing, OLED display programming, push-button handling, and alarm logic implementation in embedded systems.

During this experiment, the following concepts were learned:

- I²C communication protocol
- RTC interfacing
- OLED display handling
- Push-button interfacing
- Alarm logic implementation
- Embedded system programming

Experiment 4 (Go Further 1):

5.2 RTC-Based Water Supply Monitoring and Pump Status Display System

5.2.1 Problem Statement

The objective of this project is to design and implement an automatic motor ON/OFF control system using Arduino UNO.

The system detects water supply using a switch sensor and controls a pump indicator LED based on water availability. A DS1307 Real-Time Clock (RTC) module is used for real-time tracking and timing operations, while an SSD1306 OLED display continuously shows the current time, water supply status, and pump status.

Additionally, the system monitors whether the water supply remains available continuously for more than one hour.

5.2.2 Components Used & Pin Connections

Table 5.5: Motor ON/OFF System Components

Component	Quantity
Arduino UNO	1
DS1307 RTC Module	1
SSD1306 OLED Display	1
Push Button / Water Sensor Switch	1
LED (Pump Indicator)	1
Breadboard	1
Jumper Wires	Required

Table 5.6: Water Sensor and Pump Connections

Device	Arduino Pin
Water Supply Switch	D2
Pump Indicator LED	D3

5.2.3 RTC and OLED Connections

Table 5.7: RTC and OLED Connections

Device	Arduino Pin
RTC VCC	5V
RTC GND	GND
RTC SDA	A4
RTC SCL	A5
OLED VCC	5V
OLED GND	GND
OLED SDA	A4
OLED SCL	A5

5.2.4 Software Used

- Arduino IDE
- SimulIDE / Proteus

5.2.5 Circuit Diagram

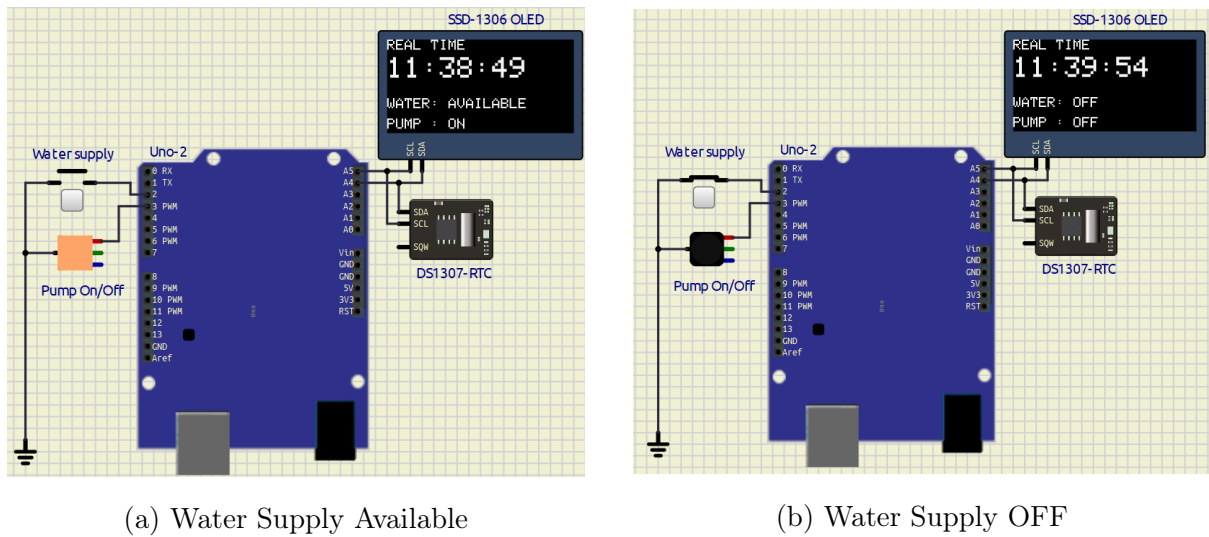


Figure 5.3: Alarm Based Motor ON/OFF System Circuit Diagrams

5.2.6 Source Code

The complete Arduino source code for this experiment is available in the project repository.

5.2.7 Working Principle

1. The water sensor continuously monitors the availability of water supply.
2. When water is available:
 - The pump indicator LED turns ON.
 - The DS1307 RTC module records the starting time.
3. The Arduino continuously calculates the elapsed time using the RTC module.
4. If the water supply remains available continuously for more than one hour:
 - The predefined timer condition becomes true.
 - Additional control logic can be implemented based on application requirements.
5. If the water supply is interrupted before one hour:
 - The system waits for two seconds.
 - The pump indicator LED turns OFF.
 - The timer is reset.
6. Throughout the operation, the SSD1306 OLED display continuously shows:
 - Current Real-Time Clock
 - Water Supply Status
 - Pump Status

5.2.8 Conclusion

The Alarm Based Motor ON/OFF System using Arduino UNO, DS1307 RTC Module, and SSD1306 OLED Display was successfully implemented and tested.

The system accurately detected water supply conditions, controlled the pump indicator based on water availability, and continuously displayed the real-time status on the OLED display. The integration of RTC timing functionality further demonstrated the implementation of time-based monitoring and automation in embedded systems.

During this experiment, the following concepts were learned:

- RTC interfacing
- OLED display interfacing
- I²C communication
- Digital input/output control
- Embedded automation logic

Experiment 4 (Go Further 2):

5.3 Temperature Data Logger Using I²C Bus

5.3.1 Problem Statement

The objective of this project is to design and implement a real-time temperature monitoring and recording system using Arduino UNO.

The system continuously measures ambient temperature using a DHT11 sensor and records the measured temperature values at regular intervals. A DS1307 Real-Time Clock (RTC) module provides accurate timestamps, while an SSD1306 OLED display continuously shows the current time, temperature, and EEPROM memory address.

The recorded temperature data is stored in the Arduino EEPROM and simultaneously transmitted to a laptop through USB serial communication for further analysis and logging.

5.3.2 Components Used & Pin Connections

Table 5.8: Temperature Data Logger Components

Component	Quantity
Arduino UNO	1
DHT11 Temperature Sensor	1
DS1307 RTC Module	1
SSD1306 OLED Display	1
Breadboard	1
USB Cable	1
Jumper Wires	Required

Table 5.9: DHT11 Sensor Connections

DHT11 Pin	Arduino Pin
DATA	D2
VCC	5V
GND	GND

5.3.3 RTC and OLED Connections

Table 5.10: RTC and OLED Connections

Device	Arduino Pin
RTC SDA	A4
RTC SCL	A5
RTC VCC	5V
RTC GND	GND
OLED SDA	A4
OLED SCL	A5
OLED VCC	5V
OLED GND	GND

5.3.4 Software Used

- Arduino IDE
- SimulIDE

5.3.5 Circuit Diagram

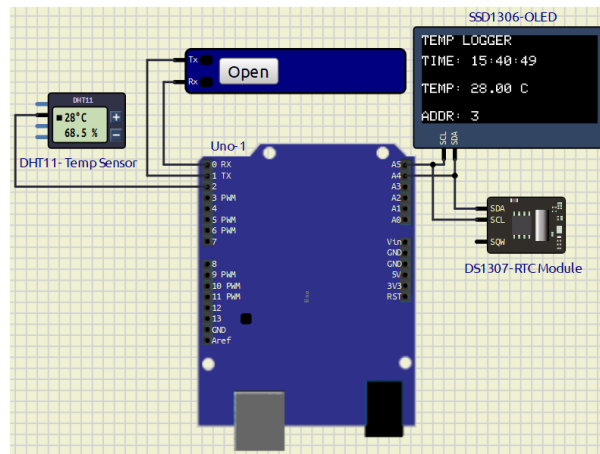


Figure 5.4: Temperature Data Logger Circuit Diagram

5.3.6 Source Code

The complete Arduino source code for this experiment is available in the project repository.

5.3.7 Working Principle

1. The DHT11 sensor continuously measures the ambient temperature.

2. The DS1307 RTC module continuously maintains the current date and time using the I²C communication protocol.
3. Arduino UNO periodically reads:
 - Current temperature from the DHT11 sensor.
 - Current time from the DS1307 RTC module.
4. The SSD1306 OLED display continuously shows:
 - Current Time
 - Current Temperature
 - EEPROM Memory Address
5. At every predefined recording interval (15 minutes), Arduino:
 - Stores the current time and temperature in the internal EEPROM.
 - Updates the EEPROM memory address.
 - Sends the recorded data to the laptop through USB serial communication.
6. The Serial Monitor continuously displays the recorded temperature log with the corresponding timestamp.
7. In this implementation, the recording interval was modified to one minute instead of fifteen minutes for demonstration purposes.

5.3.8 Output

Serial Monitor Output

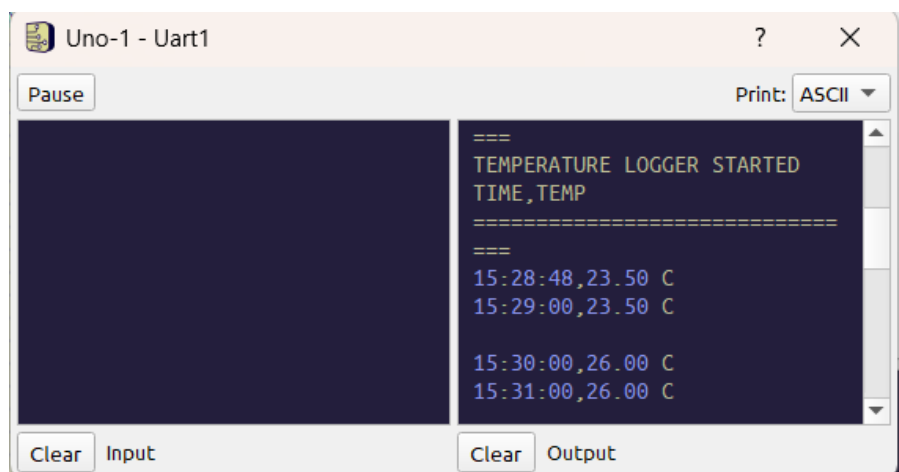


Figure 5.5: Recorded Temperature Data Displayed on the Serial Monitor

5.3.9 Conclusion

The Temperature Data Logger using Arduino UNO, DHT11 Sensor, DS1307 RTC Module, and SSD1306 OLED Display was successfully implemented and tested.

The system continuously monitored temperature, displayed live information on the OLED screen, stored periodic temperature records in the Arduino EEPROM, and transferred the recorded data to a laptop through USB serial communication. This project demonstrated an effective and low-cost embedded data logging solution.

During this experiment, the following concepts were learned:

- RTC interfacing
- OLED display interfacing
- EEPROM memory management
- Serial communication
- Temperature sensing
- Embedded data logging techniques

Bibliography

- [1] Arduino official website. [Online]. Available: <https://www.arduino.cc/>
- [2] Arduino uno rev3 documentation. [Online]. Available: <https://docs.arduino.cc/hardware/uno-rev3/>
- [3] Esp32 microcontroller series datasheet. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf
- [4] Simulide real-time circuit simulator. [Online]. Available: <https://simulide.com/>
- [5] Dht11 temperature and humidity sensor datasheet. [Online]. Available: <https://components101.com/sensors/dht11-temperature-sensor>
- [6] Ds1307 real-time clock module datasheet. [Online]. Available: <https://datasheets.maximintegrated.com/en/ds/DS1307.pdf>
- [7] Hx711 24-bit adc for load cell datasheet. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/ForceFlex/hx711_english.pdf
- [8] Load cell amplifier hx711 hookup guide. [Online]. Available: <https://learn.sparkfun.com/tutorials/load-cell-amplifier-hx711-breakout-hookup-guide>
- [9] How to set up a keypad on an arduino. [Online]. Available: <https://www.circuitbasics.com/how-to-set-up-a-keypad-on-an-arduino/>
- [10] Ssd1306 oled display driver datasheet. [Online]. Available: <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>
- [11] Sh1106 oled driver datasheet. [Online]. Available: <https://www.displayfuture.com/Display/datasheet/controller/SH1106.pdf>
- [12] 7-segment display basics. [Online]. Available: <https://www.electronics-tutorials.ws/blog/7-segment-display-tutorial.html>
- [13] 8x8 led matrix module concepts. [Online]. Available: <https://www.electronicsforu.com/resources/learn-electronics/8x8-led-matrix-module>
- [14] Adafruit ssd1306 oled display library. [Online]. Available: https://github.com/adafruit/Adafruit_SSD1306
- [15] Adafruit gfx core graphics library. [Online]. Available: <https://github.com/adafruit/Adafruit-GFX-Library>
- [16] Adafruit sh110x oled library. [Online]. Available: https://github.com/adafruit/Adafruit_SH110x
- [17] RtcLib real-time clock library. [Online]. Available: <https://github.com/adafruit/RTCLib>

- [18] Keypad library for arduino. [Online]. Available: <https://www.arduino.cc/reference/en/libraries/keypad/>
- [19] Hx711 load cell amplifier library. [Online]. Available: <https://github.com/bogde/HX711>
- [20] Lissajous curves concept. [Online]. Available: https://en.wikipedia.org/wiki/Lissajous_curve